第18课 模拟法（二）

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：报数淘汰——约瑟夫环简化版（教师演示 + 学生跟敲）

题目描述：
n 个小朋友（编号 1 到 n）围成一圈，从 1 号开始报数，每次报到 m 的人出列。输出最后剩下的那个人的编号。

输入： 一行两个整数 n 和 m（n ≤ 100，m ≤ 10）
输出： 最后剩下的人的编号

讲解思路：
1. 用数组 `alive[i]` 表示第 i 个人是否还在（1=在，0=淘汰）。
2. 用变量 `cnt` 记当前报了几个数，`left` 记还剩几个人。
3. 用 `pos` 表示当前走到了哪个位置，每次 `pos = (pos + 1) % n` 实现绕回。
4. 当 `left == 1` 时停止，找到那个 `alive[i] == 1` 的人。

```
#include <iostream>
using namespace std;

int alive[105];

int main() {
 int n, m;
 cin >> n >> m;

 // 初始化：所有人都站着
 for (int i = 0; i < n; i++) {
 alive[i] = 1;
 }

 int left = n; // 剩余人数
 int cnt = 0; // 当前报数计数
 int pos = 0; // 当前位置（从0号开始）

 while (left > 1) {
 if (alive[pos] == 1) { // 这个人还在
 cnt++;
 if (cnt == m) { // 报到 m 了，淘汰!
 alive[pos] = 0;
 left--;
 cnt = 0; // 重新开始计数
 }
 }
 pos = (pos + 1) % n; // 移到下一个位置（绕回）
 }

 // 找到最后一个还站着的人
 for (int i = 0; i < n; i++) {
 if (alive[i] == 1) {
 cout << i + 1 << endl; // 编号从1开始，所以加1
 break;
 }
 }

 return 0;
}
```

运行示例：

```
输入：8 3
输出：7
```

验证过程（板书画）：

```
8人（编号1~8，下标0~7），报到3淘汰：
第1个淘汰：3号（下标2）
第2个淘汰：6号（下标5）
第3个淘汰：1号（下标0）
第4个淘汰：5号（下标4）
第5个淘汰：2号（下标1）
第6个淘汰：8号（下标7）
第7个淘汰：4号（下标3）
最后剩下：7号（下标6）
```

教师讲解要点：

- 重点讲 `pos = (pos + 1) % n` 的作用——自动绕回
- 提问："`% n` 去掉会怎样？" pos 会一直增大，访问数组越界！
- 提问："为什么输出 `i + 1` 而不是 `i`？" 题目编号从 1 开始，数组下标从 0 开始
- 带学生手动走一遍前 3 轮，和程序输出对照

例题 2：凯撒密码加密（学生跟练）

题目描述：

输入一个只含小写字母的字符串 s 和一个整数 k（1 ≤ k ≤ 25），将字符串中每个字母向后移动 k 位（超过 z 绕回 a），输出加密后的字符串。

输入： 第一行一个字符串（长度 ≤ 100），第二行一个整数 k

输出： 加密后的字符串

讲解思路：

1. 用字符数组（或逐字符读取）存原文。
2. 对每个字母：先减 'a' 变成 0~25 的数字，加上 k，取余 26，再加 'a' 变回字母。
3. 逐个输出。

```
#include <iostream>
#include <cstring>
using namespace std;

char s[105];

int main() {
 int k;
 cin >> s >> k;

 int len = strlen(s); // 字符串长度

 for (int i = 0; i < len; i++) {
 // 第1步：字母变数字（a=0, b=1, ..., z=25）
 int num = s[i] - 'a';
 // 第2步：加上偏移量，取余绕回
 int newNum = (num + k) % 26;
 // 第3步：数字变回字母
 char newChar = newNum + 'a';
 cout << newChar;
 }
 cout << endl;

 return 0;
}
```

运行示例：

```
输入：
hello
3
输出：
khoor
```

再试一个：

```
输入：
xyz
3
输出：
abc
```

教师讲解要点：

- 分三步讲：字母数字加密计算数字字母，每步单独理解
- 重点讲 `% 26` 的作用：x(23)+3=26，26%26=0，0+'a'='a'，完美绕回！
- 让学生手动算 'y'+3：y=24，24+3=27，27%26=1，1+'a'='b'
- 提问："如果不写 `% 26`，加密 'z' 会怎样？" 25+3=28，28+'a' 不是字母了，乱码！
- **成就感设计**："你们刚刚学会了凯撒大帝的密码术！回家可以用这个方法和好朋友写秘密信哦！"

例题 3：用打印调试修复 Bug（教师演示）

题目描述：

老师故意写了一段"有 Bug 的报数淘汰程序"，和学生一起用"打印中间变量"的方法找出 Bug。

有 Bug 的代码：

```
#include <iostream>
using namespace std;

int alive[105];

int main() {
 int n, m;
 cin >> n >> m;

 for (int i = 0; i < n; i++) {
 alive[i] = 1;
 }

 int left = n;
 int cnt = 0;
 int pos = 0;

 while (left > 1) {
 if (alive[pos] == 1) {
 cnt++;
 if (cnt == m) {
 alive[pos] = 0;
 left--;
 // Bug在这里：忘记把 cnt 归零！
 }
 }
 pos = (pos + 1) % n;
 }

 for (int i = 0; i < n; i++) {
 if (alive[i] == 1) {
 cout << i + 1 << endl;
 break;
 }
 }

 return 0;
}
```

调试过程演示：

> 老师："这段代码输入 `5 2` 应该输出 `3`，但实际跑出来不对。我们怎么办？"
> "来，我们在关键位置加上 `cout`，看看每一步发生了什么——"

```
if (alive[pos] == 1) {
 cnt++;
 cout << "DEBUG: pos=" << pos << " cnt=" << cnt << endl; // 加上这句!
 if (cnt == m) {
 alive[pos] = 0;
 left--;
 cout << "DEBUG: 淘汰pos=" << pos << " left=" << left << endl; // 加上这句!
 }
}
```

> 运行后发现 `cnt` 一直在增长，报到 2 之后没有归零，变成了 3、4、5……
>
> "找到了！ `cnt` 忘记归零了！加上 `cnt = 0;` 就好了。"
>
> "看，打印中间变量就像给程序做体检，哪里不正常一眼就看出来了！"

教师讲解要点：

- 不要直接告诉答案，先让学生观察调试输出，引导他们自己发现 `cnt` 没有归零
- 强调：写模拟题的时候，一定要有意识地"检查每一步"
- 修好 Bug 之后，提醒学生删掉或注释掉调试输出

附：例题源代码与运行结果

例题 1：Josephus Problem (Count-out Game)

```
// n people (numbered 1 to n) stand in a circle. Starting from person 1,
// they count off. Whoever says m is eliminated. Output the last survivor.

#include <iostream>
using namespace std;

int alive[105];

int main() {
 int n, m;
 cin >> n >> m;

 // Initialize: everyone is alive (1 = alive, 0 = eliminated)
 for (int i = 0; i < n; i++) {
 alive[i] = 1;
 }

 int left = n; // Remaining people
 int cnt = 0; // Current count
 int pos = 0; // Current position (starting from index 0)

 while (left > 1) {
 if (alive[pos] == 1) { // This person is still alive
 cnt++;
 if (cnt == m) { // Counted to m, eliminate!
 alive[pos] = 0;
 left--;
 cnt = 0; // Reset counter
 }
 }
 pos = (pos + 1) % n; // Move to next position (wrap around)
 }

 // Find the last person standing
 for (int i = 0; i < n; i++) {
 if (alive[i] == 1) {
 cout << i + 1 << endl; // Output 1-based index
 break;
 }
 }

 return 0;
}
```

Sample Input

```
8 3
```

Expected Output

```
7
```

例题 2：Caesar Cipher Encryption

```
// Input a lowercase string s and an integer k (1 <= k <= 25).
// Shift each letter forward by k positions (wrap around from z to a).

/*
#include <iostream>
#include <cstring>
using namespace std;

char s[105];

int main() {
 int k;
 cin >> s >> k;

 int len = strlen(s); // Get string length

 for (int i = 0; i < len; i++) {
 // Step 1: Convert letter to number (a=0, b=1, ..., z=25)
 int num = s[i] - 'a';
 // Step 2: Add offset and wrap around using modulo
 int newNum = (num + k) % 26;
 // Step 3: Convert number back to letter
 char newChar = newNum + 'a';
 cout << newChar;
 }
 cout << endl;

 return 0;
}
*/
```

Sample Input

```
hello
3
```

Expected Output

```
khoor
```

例题 3：Debugging Demo - Buggy Josephus Code

```
// This example shows a buggy version where cnt is NOT reset after elimination.
// Students learn to add debug prints to find bugs.
// The bug: missing "cnt = 0;" after eliminating a person.

/*
#include <iostream>
using namespace std;

int alive[105];

int main() {
 int n, m;
 cin >> n >> m;

 for (int i = 0; i < n; i++) {
 alive[i] = 1;
 }

 int left = n;
 int cnt = 0;
 int pos = 0;

 while (left > 1) {
 if (alive[pos] == 1) {
 cnt++;
 // DEBUG: print current state to find the bug
 cout << "DEBUG: pos=" << pos << " cnt=" << cnt << endl;
 if (cnt == m) {
 alive[pos] = 0;
 left--;
 cout << "DEBUG: eliminated pos=" << pos << " left=" << left << endl;
 // BUG: forgot to reset cnt to 0 here!
 // Fix: uncomment the line below
 // cnt = 0;
 }
 }
 pos = (pos + 1) % n;
 }

 for (int i = 0; i < n; i++) {
 if (alive[i] == 1) {
 cout << i + 1 << endl;
 break;
 }
 }

 return 0;
}
*/
```

Sample Input

```
5 2

Expected Output (with bug fix):
3

Note: This example is for debugging demonstration.
The DEBUG lines show how cnt keeps growing without being reset.
Adding "cnt = 0;" after the elimination fixes the bug.
```